



Object-oriented programming

You're well on your way now! In this video, you'll learn more about Python and what makes it both approachable and powerful as a coding language. You're going to learn about object-oriented programming. Specifically, we're going to discuss classes, methods, and attributes. What follows is a brief introduction to object-oriented programming. A more detailed discussion is beyond the scope of this course. You'll have the opportunity to further explore object-oriented programming in your future career as a data professional.

Object-oriented programming is a programming system that is based around objects, which can contain both data and useful code that manipulates that data. An object is an instance of a class. Think of it like a fundamental building block of Python. Lists, functions, strings... these are all objects. The main idea behind object-oriented programming is to have both data and the methods that manipulate it within objects, allowing for more organized, accessible, and reusable code. The most important concept in object-oriented programming is the class. A class is an object's data type that bundles data and functionality together.

In other words, the reason why it's useful for an object to have a type – or belong to a class – is because it allows us to build a bunch of useful tools that can be packed

directly into the object itself. This will make a lot more sense with an example. When we put the words "hocus pocus" inside quotations and assign that to a variable called "magic," this variable becomes an instance of the string class. Because it belongs to the string class, it behaves in a certain way and has a lot of built-in functionality reserved for strings. We can swap the case of the characters by typing "magic dot swapcase" with empty parentheses after it. We can replace some characters with new characters by typing "magic dot replace" and entering the characters we want to replace and what we want to replace them with. And we can split the string into a list of two strings using "dot split" and an empty pair of parentheses.

These actions are known as methods. A method is a function that belongs to a class and typically performs an action or operation. They use parentheses. In our examples, each method acted on the value of our variable. It changed it in some way. That's what I mean when I say that methods typically perform an action or operation. By the way, you don't have to memorize these methods.

There aren't many people who know all of them. Most coding environments have ways to access a list of methods available to a given class. In Jupyter Notebook, we type a dot and then hit the tab key. Notice that we're attaching the method to its class instance using a dot. That is called dot notation, and it's how we access the methods and attributes that belong to an instance of a class. There are many different classes in Python. You've encountered some of these already.

The core classes of Python are: Integers, floats, strings, booleans, lists, dictionaries, tuples, sets, frozensets, functions, ranges, and none, which is a data type that returns an empty value. There are also many additional custom-defined classes that come with

libraries and packages, and you can even make your own. Okay, the last thing we're going to discuss is attributes. Attributes are values associated with an object or class which are referenced by name using dot notation. They don't use parentheses.

Attributes are especially important for custom-built classes and more complex data structures, like dataframes. You'll learn more about these later, but here's an example.

Suppose we have a dataframe called "planets" that contains a row for each planet and columns that represent planet name, its radius, and the number of moons it has. One attribute of this dataframe would be its shape. This dataframe is eight rows by three columns. Another attribute of the dataframe class is columns. Calling this attribute on the dataframe object returns an index object containing the column names of the dataframe. Attributes allow you to access characteristics of a class, but they don't do anything to it or change it. Perhaps you're beginning to appreciate how object-oriented programming is an ideal structure for data analysis.

By packaging data together with ways to manipulate it and learn about it, objects are the fundamental building blocks of Python and part of what makes it such a powerful tool for data professionals.

Object-Oriented Programming

A programming system that is based around objects, which can contain both data and code that manipulates that data

Object

An instance of a class; a fundamental building block of Python

Class

A class is an object's data type that bundles data and functionality together

Method

A function that belongs to a class and typically performs an action or operation

Dot Notation

How to access the methods and attributes that belong to an instance of a class

Core Python Classes

- Integers
- Floats
- Strings
- Booleans
- Lists
- Dictionaries
- Tuples
- Sets
- Frozensets
- Functions
- Ranges
- None
- Custom-defined

Attribute

A value associated with an object or class which is referenced by name using dot notation

Question



There are four fundamental concepts in object-oriented programming, including objects. What are the other three concepts? Select all that apply.

☐ Operators

☒ Classes

✔ **Correct**

The four fundamental concepts in object-oriented programming include objects, classes, attributes, and methods.

☒ Methods

✔ **Correct**

The four fundamental concepts in object-oriented programming include objects, classes, attributes, and methods.

☒ Attributes

✔ **Correct**

The four fundamental concepts in object-oriented programming include objects, classes, attributes, and methods.

Continue

Skip